Progressive Retrieval Attention with Ollama: Transparent Context Optimization for Local-Model Applications

Jorge Simão

September 2026

Abstract

Ollama combines local-model packaging, lifecycle management, and a stable application API. Progressive Retrieval Attention (PRA) should preserve that product surface: ordinary installations receive selected-text E0, while a PRA-aware model backend may negotiate native E2 without requiring a second attention implementation in Ollama. We implement this capability-negotiated adapter, pin a current source audit, and test an Ollama 0.6.8 daemon on Windows. Across 15 natural-QA examples, selected prompts used 37.5–51.7% of full-context tokens. On HotpotQA and QASPER, median time to first token decreased from 602.8 to 348.6 ms and from 732.7 to 300.0 ms, respectively; 2Wiki reversed direction. Keeping Qwen2.5-0.5B alive reduced mean request latency from a 2300.5 ms cold load to 506.5 ms across five warm requests (4.54×). No native backend handshake was available, so AUTO correctly remained E0. The result is a usable gateway integration and a precise delegation boundary, not an Ollama-native E2 claim.

1 Introduction

Ollama's value is operational simplicity: a model name, one daemon, and a familiar API. A PRA integration that requires every application to understand engine-specific K/V state would undermine that value. We instead treat Ollama as the product and model-lifecycle layer. PRA owns typed records, authorization, selection, sessions, and storage policy. The active model runner owns attention and physical cache state.

This division creates one hard rule: deeper support is negotiated, never inferred from a version string or from Ollama's historical relation to a particular backend. Current Ollama may select different runners for different models and platforms. An application-facing adapter must therefore be able to downgrade safely.

Our contributions are an Ollama-native HTTP adapter, a model-fingerprint invalidation rule, optional backend delegation, live selected-text and lifecycle measurements, and tests that keep the advertised integration level honest.

2 PRA Levels and Ownership

E0 serializes selected evidence into visible messages. E1 transports logical resource identities without changing attention. E2 consumes detached selected native K/V. E3 additionally requires scheduler-owned placement, sharing, prefetch, and eviction. The adapter reports the deepest implemented level, not the deepest level described by the protocol.

3 Pinned Architecture Audit

The source audit pins Ollama commit `e37a00a8fa94fca07cefd544bffc9d8997ebcd44`. The measured daemon is version 0.6.8; these are recorded separately because behavior observed in one must not be projected onto the other. The audited code exposes chat/generate, model inspection, installed/running model lists, keep-alive, and runner scheduling. Backend choice is an implementation decision, so the PRA capability cannot be deduced from the model package alone.

The public API exposes useful timing counters: load, prompt evaluation, token evaluation, and total duration. It does not expose a detached semantic K/V contract. Thus the stock daemon qualifies at E0. A future Ollama integration can reach E2 by delegating to a backend adapter with model/resource identity and native attention support; it should not duplicate backend kernels in the product layer.

Existing Ollama client → PRA gateway/SDK → capability probe.

No native handshake → selected records become an ordinary system message → `/api/chat` (E0).

Validated backend handshake → resource/version/model fingerprint → backend-native PRA executor (E2/E3).

Unload or model switch → invalidate incompatible native state; Ollama continues to own runner load, keep-alive, and scheduling.

Figure 1: Gateway-first integration. Native support is inherited from the active backend rather than reimplemented in Ollama.

Table 1: Natural E0 qualification on Ollama. Prompt ratio is selected/full; TTFT is median ms; F1 is selected/full.

Dataset	Prompt ratio	TTFT selected/full	F1 selected/full
HotpotQA	0.410	348.6 / 602.8	0.404 / 0.314
QASPER	0.375	300.0 / 732.7	0.217 / 0.099
2Wiki	0.517	309.2 / 263.3	0.036 / 0.136

4 Implementation

`OllamaEngineAdapter` calls `/api/show` before execution and hashes the model name, modification metadata, details, and model information. A model change invalidates state held by an optional native backend executor. In E0, selected resources are inserted as an ordinary system message with explicit resource IDs and URIs, then sent to `/api/chat`. Tool schemas and the request's generation bound are preserved.

`inspect_endpoint()` queries `/api/version`, `/api/tags`, and `/api/ps`. The runtime provider's doctor reports the endpoint and models but labels native PRA as fallback unless a backend handshake is supplied. `unload()` uses Ollama's zero keep-alive request and invalidates delegated state. Unit tests cover E0 injection, endpoint inspection, metrics preservation, and the rule that only explicit backend delegation upgrades to E2.

5 Experimental Method

We ran Ollama 0.6.8 on Windows 10 with Qwen2.5-0.5B. The natural cohort freezes the evidence selection and compares no context, selected text up to 384 source tokens, and full text up to 1024 source tokens. It contains five examples each from HotpotQA, QASPER, and 2WikiMultiHopQA, with two repeats and 16 generated tokens. Streaming HTTP measurements provide TTFT and inter-token latency.

The lifecycle experiment unloads Qwen, runs one cold and five keep-alive requests, switches to SmolLM2-135M, unloads it, and returns to Qwen. The selected record and query remain fixed. This isolates model lifecycle from retrieval. The cohort is a smoke calibration and does not support production tail-latency or model-quality claims.

6 Results

Selected text substantially reduces visible context. It also improves TTFT and token F1 on HotpotQA and QASPER in this run. The 2Wiki reversal is equally important: the frozen selector's evidence and a small-model generation cap are not uniformly sufficient. E0 input compression is measured; universal quality preservation is not.

The warm mean is 4.54× faster than the initial cold request. Most of the difference is model load and prompt evaluation, not PRA logic. The switch test also shows why the adapter binds native state to a model fingerprint. Returning to Qwen was warm in this daemon configuration, but an adapter cannot assume that a runner survives every memory-pressure or scheduling event.

7 Deployment Matrix

This design keeps ordinary clients compatible. An application can point the PRA gateway at Ollama today. If a future active backend reports E2, the same wire request can carry stable identities while the backend performs native materialization. If the handshake disappears after an upgrade or model switch, AUTO returns to E0.

Table 2: Lifecycle measurements. Warm is the mean of five keep-alive requests.

Event	Total ms	Load ms	Prompt-eval ms
Qwen cold after unload	2300.5	1500.6	329.9
Qwen warm mean	506.5	30.4	5.9
Switch to SmolLM2	5248.7	4465.3	248.3
Return to Qwen	503.2	36.1	5.7

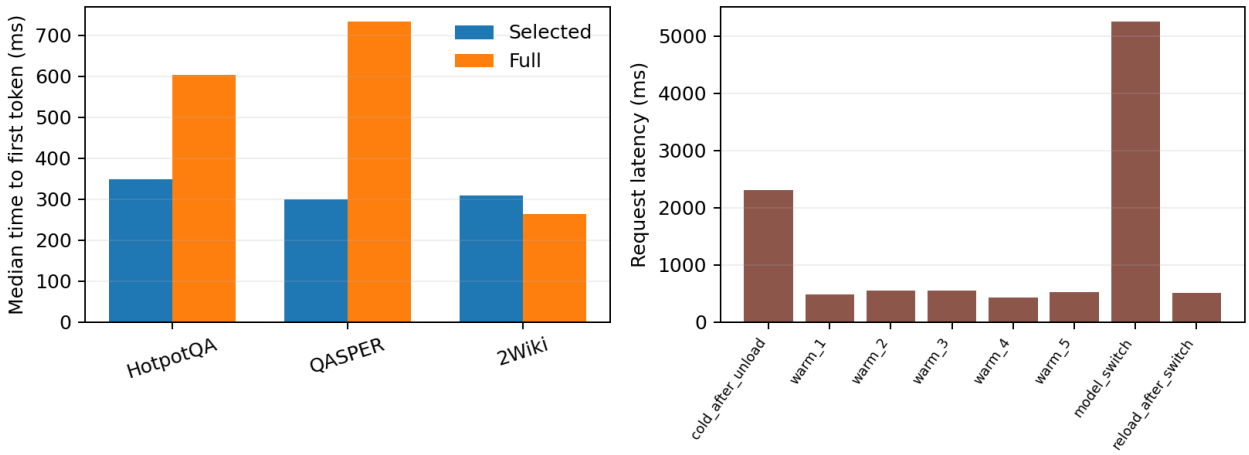

Figure 2: Selected/full context pressure (left) and model lifecycle (right). Ollama's keep-alive benefit is large enough that lifecycle state must be reported separately from PRA selection.

8 Limitations and Next Work

The local runtime is old relative to the pinned source audit; the paper does not merge those observations. Only one main model, one host, and a small cohort were measured. Memory use, concurrent requests, cancellation, reconnects, daemon restart, quantized variants, and native E0/E2 parity remain open. The next step is to expose the same capability handshake through a PRA-aware llama.cpp or other active backend, then rerun frozen-selection quality and lifecycle tests without changing the client.

9 Conclusion

Ollama can provide a low-friction PRA entry point without becoming a second PRA runtime. The measured E0 path reduces visible context and benefits two of three small natural-QA slices; keep-alive dominates cold latency. Native support must be delegated and negotiated. On the tested daemon it was absent, and the safe fallback worked as designed.

Artifact Availability

Branch `research/paper6-8-ollama` contains the adapter, tests, raw JSON, lifecycle benchmark, plot generator, README, and manuscript.

References

[1] Ollama contributors. *Ollama*. https://github.com/ollama/ollama

[2] OpenAI. *Chat Completions API*. https://platform.openai.com/docs/api-reference/chat

[3] Qwen Team. *Qwen2.5 Technical Report*. 2024. https://arxiv.org/abs/2412.15115

Table 3: Current product status.

Mode	Transport	Status	Claim
E0 selected	Ollama/OpenAI HTTP	measured	supported default
E1 logical	optional envelope	interface only	no native effect
E2 native	delegated backend	not negotiated	AUTO falls back
E3 scheduler	Ollama runner	not implemented	no claim